

Actividad Guiada: GitHub y Git. Primeros pasos

Descargar

- [Actividad Guiada: GitHub y Git. Primeros pasos](#)
 - ▼ [Parte 1: Configuración del Entorno de Trabajo en GitHub](#)
 - [Actividad 1. Creación de organizaciones y equipos de GitHub de forma individual](#)
 - ▼ [Parte 2: Trabajando en Local con el repositorio creado en GitHub, mediante comandos de Git](#)
 - [Actividad 2. Trabajo en local con Git](#)
 - ▼ [Anexo](#)
 - [A.1 Glosario de Comandos Utilizados](#)
 - [A.2 Flujo recomendado para el día a día](#)

En esta actividad aprenderéis el flujo básico de trabajo con Git y GitHub: clonar un repositorio, crear y usar ramas, guardar cambios con commits y sincronizar correctamente vuestro trabajo entre el entorno local y el repositorio remoto.

Herramientas:

- **Navegador web** (para GitHub)
- **Visual Studio Code** (para los comandos de Git en local)

Parte 1: Configuración del Entorno de Trabajo en GitHub

Vamos a hacer una primera aproximación al concepto de Organización, Equipo y Repositorio en GitHub. Para ello vamos a realizar primero una actividad individual que permita a todos los alumnos pasar por todos los pasos y luego los volveremos a repetir por equipos para que tenga más sentido.

- **¿Qué es una organización en GitHub?**

Una **organización** en GitHub es un espacio compartido donde un equipo o empresa puede gestionar múltiples repositorios juntos. A diferencia de un usuario personal, una organización puede tener varios propietarios y miembros con distintos niveles de acceso.

Las organizaciones incluyen funcionalidades adicionales como equipos (*teams*), gestión de permisos por repositorio, y en los planes de pago, herramientas de seguridad avanzadas.

- **¿Cuándo usar usuario personal u organización?**

Situación	Recomendación
Proyecto personal, portfolio, aprendizaje	Usuario personal
Proyecto de empresa o startup	Organización
Proyecto open source con varios colaboradores	Organización
Trabajo freelance en solitario	Usuario personal
Equipo de desarrollo (2 o más personas)	Organización

Info

Si trabajas solo o en proyectos personales, el usuario personal es suficiente. En cuanto haya un equipo, una empresa o varios proyectos relacionados, una organización facilita mucho la gestión.

Actividad 1. Creación de organizaciones y equipos de GitHub de forma individual

Paso 1: Alta en GitHub si aún no tenéis cuenta

Cada miembro del equipo debe tener su propia cuenta de GitHub.

1. Id a github.com.
2. Haced clic en **"Sign up"** y seguid las instrucciones para crear una cuenta gratuita.

Paso 2: Crear una Organización

La organización en GitHub servirá como el contenedor de vuestro proyecto de equipo cuando comencemos con el proyecto, ahora se trata de que todos adquiráis los conocimientos necesarios para realizar estas acciones, por lo que:

1. Cada uno de vosotros deberá crear una organización.
2. Haced clic en el icono **+** en la esquina superior derecha de GitHub y seleccionad **"New organization"**.
3. Elegid el plan gratuito ("Free").
4. Nombrad la organización siguiendo este formato: `[apellido1_nombre]_pi2damiesbalmis` . Por ejemplo: `garcia_sonia_pi2damiesbalmis` .
5. Completad los pasos de creación.

Paso 3: Añadir Miembros a la Organización

Ahora, el propietario de la organización debe invitar al menos a tres compañeros y al **profesor**.

1. Dentro de la página de la organización, id a la pestaña **"People"**.
2. Haced clic en el botón **"Invite member"**.
3. Buscad por nombre de usuario de GitHub o email a cada compañero y al profesor y enviad las invitaciones.
4. Los invitados recibirán un correo para aceptar la invitación. ¡Aceptadla!
5. Cambiar el rol del profesor a **"Owner"** para que tenga permisos completos.

Paso 4: Crear un Equipo

Los equipos ayudan a gestionar los permisos de acceso a los repositorios.

Note

Crear un equipo es algo opcional en este caso, pero es una buena práctica para gestionar permisos en proyectos más grandes.

1. En la página de la organización, id a la pestaña **"Teams"**.
2. Haced clic en **"New team"**.
3. Dadle un nombre al equipo, por ejemplo, **"equipo1_garcia_sonia"**.
4. En el siguiente paso, añadid a todos los miembros de la organización al equipo.

Paso 5: Crear un Repositorio de prueba

1. En la página principal de la organización, haced clic en el botón verde **"New"** para crear un nuevo repositorio.
2. **Repository name:** pruebas
3. **Description:** (Opcional) "Repositorio para practicar el flujo de Git".
4. Seleccionad **"Public"**.
5. **Importante:** Marcad la casilla **"Add a README file"**. Esto inicializa el repositorio con un primer fichero.
6. Haced clic en **"Create repository"**.

Parte 2: Trabajando en Local con el repositorio creado en GitHub, mediante comandos de Git

En este punto vamos a clonar el repositorio de GitHub a una carpeta local de nuestro ordenador. Crearemos una nueva rama aparte de la `main` para acostumbrarnos a realizar el trabajo de forma organizada.

Actividad 2. Trabajo en local con Git

- `git clone` crea una copia completa de un repositorio remoto en tu máquina, incluyendo todo el historial.

```
# Clonar un repositorio de GitHub
git clone https://github.com/usuario/repositorio.git

# Clonar en una carpeta con un nombre diferente
git clone https://github.com/usuario/repositorio.git mi-carpeta

# Clonar solo la rama principal (más rápido en repos muy grandes)
git clone --single-branch https://github.com/usuario/repositorio.git

# Clonar y entrar en la carpeta
git clone https://github.com/usuario/repositorio.git
cd repositorio
```

Info

Al clonar, Git configura automáticamente `origin` apuntando al repositorio de GitHub.

Paso 1: Clonar el Repositorio

1. En Visual Studio Code abriremos la carpeta donde vayamos a situar la carpeta de nuestro repositorio.
2. **Clonar el repositorio:**
 - En la página del repositorio `pruebas` en GitHub, haced clic en el botón verde `[<> Code]`.

- Copiad la URL HTTPS (por ejemplo:
`https://github.com/nombre-organizacion/pruebas.git`).
- En la terminal bash de VS Code, ejecutar:

```
git clone [URL_COPIADA_DEL_REPOSITORIO]
```

- Abrir en VS Code la nueva carpeta creada. O situarte en ella en la terminal con:

```
cd pruebas
```

Las ramas (*branches*) son una de las características más poderosas de Git. Permiten trabajar en nuevas funcionalidades, correcciones o experimentos de forma completamente aislada del código principal, y fusionarlos cuando estén listos. En un repositorio de un equipo cada persona debe trabajar en su propia rama para no interferir con el trabajo de los demás. Una rama es como una línea temporal independiente del proyecto.

- `git branch` - Gestionar ramas

```
# Ver todas las ramas locales
git branch

# Ver todas las ramas (locales y remotas)
git branch -a

# Crear una nueva rama (sin cambiarte a ella)
git branch nombre-rama

# Eliminar una rama (solo si ya fue fusionada)
git branch -d nombre-rama

# Eliminar una rama forzosamente (aunque no haya sido fusionada)
git branch -D nombre-rama

# Renombrar la rama actual
git branch -m nuevo-nombre

# Ver la última actividad de cada rama
git branch -v
```

- `git switch` - Cambiar de rama (comando moderno)
`git switch` es el comando moderno para cambiar entre ramas (disponible desde Git 2.23).

```
# Cambiar a una rama existente
git switch nombre-rama

# Crear una nueva rama y cambiarte a ella en un solo paso
git switch -c nueva-funcionalidad

# Volver a la rama anterior
git switch -
```

- **git checkout** - Cambiar de rama (comando clásico)
git checkout es el comando clásico. Funciona igual que **git switch** pero también tiene otros usos (como restaurar archivos).

```
# Cambiar a una rama existente
git checkout nombre-rama

# Crear una nueva rama y cambiarte a ella
git checkout -b nueva-funcionalidad

# Volver a la rama anterior
git checkout -
```

Info

switch vs **checkout** : ambos funcionan igual para cambiar de rama. **switch** es más moderno y claro en su propósito. En proyectos actuales se recomienda **switch** , pero encontrarás **checkout** en mucha documentación y tutoriales.

Paso 2: Crear y Moverse a una Rama Personal

1. **Crear una nueva rama:** Reemplazad **[tu-nombre]** por vuestro nombre en minúsculas.

```
git branch [tu-nombre]
```

2. **Moverse a la nueva rama:**

```
git switch [tu-nombre]
```

Info

Acabáis de crear una "copia" del estado actual de la rama `main` y os habéis movido a ella para empezar a trabajar. Todos vuestros cambios a partir de ahora se harán en esta nueva rama.

Paso 3: Crear y Añadir un Fichero

Ahora, añadiremos un fichero al proyecto.

1. **Crear un fichero Markdown:** Usad el Bloc de notas o vuestro editor preferido para crear un fichero con vuestro nombre. Por ejemplo, `juan.md`.

```
notepad juan.md
```

2. Añadir contenido al fichero. Podéis usar sintaxis Markdown, por ejemplo:

```
# Fichero de Juan

* Me gusta el desarrollo de software.
* Mi lenguaje favorito es Python.
```

3. Guardar y cerrad el fichero.
4. **Añadir el fichero al "staging area":** Le indicamos a Git que queremos incluir este fichero en nuestro próximo "guardado" (commit).

```
git add juan.md
```

5. **Hacer commit:** Guardamos los cambios de forma permanente en el historial de nuestra rama.

```
git commit -m "Añade fichero de Juan"
```

- `git push` - Subir cambios al remoto

`git push` envía los commits de tu repositorio local al repositorio remoto.

```
# Subir la rama actual al remoto (primera vez, con -u para configurar el seguimiento)
git push -u origin main

# Subir la rama actual (después de haber configurado el seguimiento con -u)
git push

# Subir una rama específica
git push origin nombre-rama

# Subir todas las ramas locales
git push --all origin
```



Importante

Solo puedes hacer `push` si no hay cambios en el remoto que tú no tengas. Si alguien más ha subido algo, primero debes hacer `git pull`.

Paso 4: Subir la Rama a GitHub

Vuestros cambios (la nueva rama y el nuevo fichero) solo existen en vuestro ordenador. Ahora los subiréis a GitHub.

```
git push --set-upstream origin [tu-nombre]
```



Note

El comando `--set-upstream origin [tu-nombre]` crea un enlace entre tu rama local y una nueva rama con el mismo nombre en el repositorio remoto (GitHub), para que en futuros `push` solo necesites escribir `git push`.

Paso 5: Añadir nuevos cambios a tu rama y actualizar

Crea un nuevo fichero llamado `saludo.md` en tu rama personal, añade un saludo gracioso, súbelo a GitHub y luego integra esos cambios en `main` para que queden iguales en local y en remoto.

1. Asegúrate de estar en tu rama personal:

```
git switch [tu-nombre]
```

2. Crea el fichero `saludo.md` con un saludo gracioso:

```
notepad saludo.md
```

Escribe algo como este ejemplo y guarda:

```
# Saludo del equipo
```

```
Hola profe. Si este commit falla, echadle la culpa al teclado, no al programador.
```

3. Añade, confirma y sube los cambios de tu rama:

```
git add saludo.md
git commit -m "Añade saludo gracioso en saludo.md"
git push origin [tu-nombre]
```

4. Cámbiate a `main` y actualízala con lo último del remoto:

```
git switch main
git pull origin main
```

5. Comprueba qué hay en `main` antes de integrar tu rama:



Note

Si los cambios anteriores no existen en `main`, es normal: todavía no has integrado tu rama personal.

- `git merge` - Fusionar ramas

`git merge` integra los cambios de una rama en otra. Debes estar en la rama de destino antes de ejecutarlo.

```
# Primero, cámbiate a la rama que recibirá los cambios
git switch main

# Fusiona la rama de la funcionalidad en main
git merge rama-personal

# Fusionar sin crear un commit de merge (si es posible)
git merge --ff-only rama-personal

# Fusionar creando siempre un commit de merge (mantiene el historial)
git merge --no-ff rama-personal
```

Paso 6: Integrar tu rama a main

Integra tu rama personal en `main` y sube `main` a remoto:

1. Integra tu rama a main, añadiendo los nuevos cambios:

```
git merge [tu-nombre]
git push origin main
```

2. Comprueba que `saludo.md` ya existe en `main` y tiene el mismo contenido.
3. Verifica que no tienes cambios pendientes en local:

```
git status
```

4. Sincroniza referencias y compara `main` local con `origin/main`:

```
git fetch origin
git diff main origin/main
```



Importante

Si `git diff main origin/main` no muestra nada, significa que `main` está igual en local y en remoto.

5. **Comprueba en GitHub web** que en la rama `main` aparece el fichero `saludo.md` con el contenido que has creado.



Info

Si trabajas tú solo en el repositorio, puedes usar ramas para organizar tareas concretas y eliminarlas cuando ya estén integradas en `main`. En ese escenario, la integración suele ser sencilla y sin conflictos.

Paso 7: Trabajar en paralelo

1. Crea ramas para diferentes funcionalidades:

```
git branch funcionalidad-carrito
git branch funcionalidad-busqueda

# Verifica que se crearon
git branch
```

2. Trabaja en las distintas ramas en paralelo. Por ejemplo, creando archivos con distintos contenidos:

```
# Trabaja en la rama del carrito
git switch funcionalidad-carrito
echo "<div id='carrito'>0 artículos</div>" >> index.html
git add index.html
git commit -m "Añade icono del carrito de compras"

# Cambia a la rama de búsqueda y añade otro cambio independiente
git switch funcionalidad-busqueda
echo "# Mejora de búsqueda" > busqueda.md
echo "Añadir campo de búsqueda en la cabecera" >> busqueda.md
git add busqueda.md
git commit -m "Añade propuesta inicial para funcionalidad de búsqueda"

# Vuelve a main y trabaja en otra cosa sin interferir
git switch main
echo "/* Nuevos colores corporativos */" >> css/styles.css
git add css/styles.css
git commit -m "Actualiza paleta de colores corporativos"

# Verifica que main no tiene los cambios del carrito
cat index.html # El div del carrito no aparece aquí
```

3. Fusionar cambios:

```
# Asegúrate de estar en main
git switch main

# Fusiona la rama del carrito
git merge funcionalidad-carrito

# Verifica que los cambios del carrito están ahora en main
cat index.html # Ahora sí aparece el div del carrito

# El historial muestra ambas líneas de desarrollo
git log --oneline --graph

# Elimina la rama ya fusionada porque se ha terminado el trabajo en ella
git branch -d funcionalidad-carrito
```

Info

En la siguiente actividad simularemos un caso más real, con varias personas trabajando a la vez sobre el mismo repositorio, donde sí pueden aparecer conflictos y errores de sincronización.

Anexo

A.1 Glosario de Comandos Utilizados

Comando	Descripción en esta actividad
<code>git config --global user.name</code>	Configura tu nombre de autor para todos tus repositorios.
<code>git config --global user.email</code>	Configura tu email de autor para todos tus repositorios.
<code>git clone [url]</code>	Crea una copia local de un repositorio remoto de GitHub.
<code>git branch [nombre-rama]</code>	Crea una nueva rama.
<code>git switch [nombre-rama]</code>	Cambia a la rama indicada para trabajar en ella.
<code>git checkout [nombre-rama]</code>	Se mueve a la rama especificada para empezar a trabajar en ella.
<code>git add [fichero]</code>	Añade un fichero modificado al "área de preparación" (staging) para incluirlo en el próximo commit.
<code>git commit -m "mensaje"</code>	Guarda una instantánea de los cambios preparados en el historial local de la rama.
<code>git push origin [nombre-rama]</code>	Sube los commits de tu rama local a la rama correspondiente en GitHub (remoto <code>origin</code>).
<code>git pull origin [nombre-rama]</code>	Descarga los cambios de una rama remota y los fusiona en tu rama local actual.
<code>git fetch origin</code>	Actualiza las referencias remotas sin fusionarlas automáticamente en tu rama local.
<code>git diff main origin/main</code>	Compara la rama <code>main</code> local con <code>origin/main</code> para verificar si están iguales.
<code>git status</code>	Muestra si hay cambios pendientes o si el repositorio está limpio.
<code>git merge [nombre-rama]</code>	Fusiona el historial de la rama especificada en la rama actual.

A.2 Flujo recomendado para el día a día

Este es el flujo de trabajo que se recomienda seguir en el trabajo diario con Git y GitHub:

Al empezar la jornada

```
git pull           # 1. Descarga los cambios del equipo
git status         # 2. Confirma que todo está limpio
git switch -c tarea/descripcion # 3. Crea una rama para la nueva tarea
```

Durante el trabajo

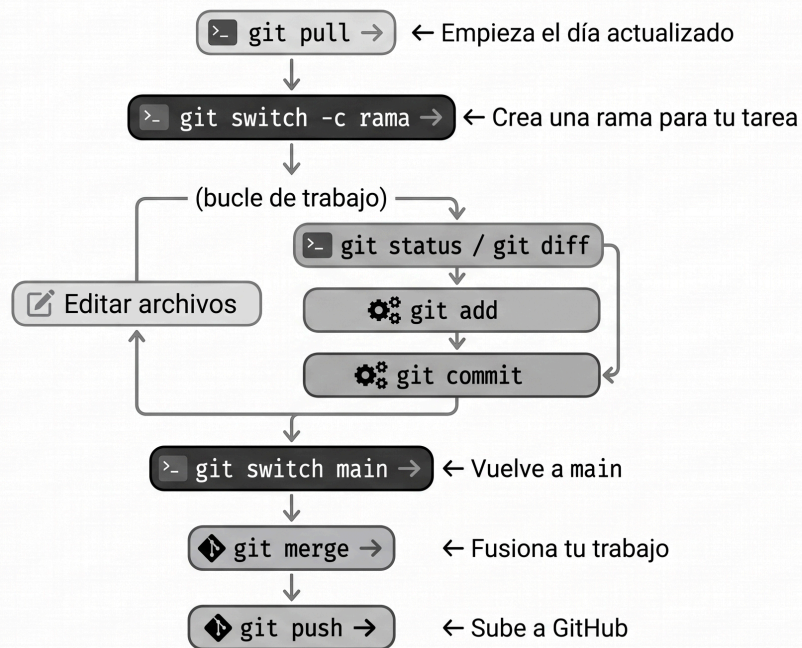
```
# Trabaja en tu código...

git status         # Revisa qué ha cambiado
git diff          # Revisa los detalles del cambio
git add .         # Prepara los cambios
git commit -m "Descripción clara" # Guarda el punto de control
# Repite: trabaja → status → add → commit
```

Al terminar la tarea

```
git switch main    # Vuelve a la rama principal
git pull           # Descarga posibles cambios nuevos
git merge tarea/descripcion # Fusiona tu trabajo
git push           # Sube al repositorio remoto
git branch -d tarea/descripcion # Limpia la rama ya fusionada
```

Diagrama del flujo completo



⚠ Regla de oro

Haz **pull** antes de empezar y **push** al terminar. Haz commits pequeños y frecuentes. Usa ramas para todo lo que no sea un cambio trivial.